COLEGIO UNIVERSITARIO IES

Aplicaciones web 2

Analista de Sistemas y Desarrollo de Software

Proyecto Web - "Pizzería Lobato"

Informe de desarrollo

Integrantes

Mariano Alberto López
Facundo Elgueta Millán

INDICE

1. Información recolectada (TP1) 2

2. Equipo y roles (TP1) 3

3. ¿Qué se va a desarrollar? (TP1) 3

4. Cronograma y organización de tareas (TP1) 4

5. Seguimiento y control de versiones (TP1) 5

6. Back-end para servir el sitio web (TP2) 6

1. Información recolectada (TP1)

Ficha

● **Nombre**: Pizzería Lobato

● **Tipo de negocio**: Gastronómico – Elaboración y venta de pizzas y empanadas

● **Ubicación**: Buenos Aires, Argentina

● **Contacto:** Tel: +54 011 46418269

● **Persona designada para brindar información:** Sebastián Maciel – Correo: maciel_77@live.com.ar – Tel: +54 011 37931972

Sobre Pizzería Lobato

Pizzería Lobato es una reconocida marca de barrio que ya cuenta con clientes fieles desde sus inicios.

Nace con Felipe Lobato que era conocido en el barrio como el mejor pizzero de la zona de La Matanza. Luego de su muerte en 1973, sus tres hijos decidieron seguir con el legado de su papá, abriendo en ese mismo año "Pizzería Lobato", donde comenzó siendo una simple pizzería de barrio y terminó siendo una de las más concurridas y recomendadas.

Pasados los años, como todo negocio, tuvo que renovarse por las inevitables innovaciones de la sociedad y cambios económicos del país. Por lo que dos de sus hijos se fueron a España donde continúan cocinando para la gente en un pueblo chico de Madrid. Quedando así solo Sebastián, que continúa con el legado de su padre para que el barrio y todo el partido de La Matanza no se queden sin las clásicas pizzas Lobato.

Lista de requerimientos funcionales

● Página de inicio con una breve presentación de la pizzería, mostrar algunos de los productos y promociones con imágenes destacadas.

● Mostrar a través de la pagina de inicio la dirección de la pizzería y la posibilidad de acceder directamente a las promociones y carta.

● Formulario de contacto con validación de campos.

● Diseño responsivo para desktop y móvil.

● Menú de navegación fijo y accesible.

● Sección de Carta interactiva con filtros.

● Animaciones suaves en scroll y hover para mejorar la experiencia.

2. Equipo y roles (TP1)

❖ **López Mariano**: Líder de proyecto.

❖ **Elgueta Facundo**: Desarrollador Fullstack.

**** Al contar con solo dos integrantes los roles se pueden ir intercalando a medida que avance el proyecto.***

3. ¿Qué se va a desarrollar? (TP1)

Actualmente se ha estado desarrollando una página “estática”, con esto queremos decir que el cliente solamente puede interactuar con la carta, pero no hay envió ni recibimiento de datos.

Lo que nosotros queremos implementar es un “**carrito de compras**” para permitir los pedidos vía web y así, poder agilizar y facilitar la compra de los clientes.

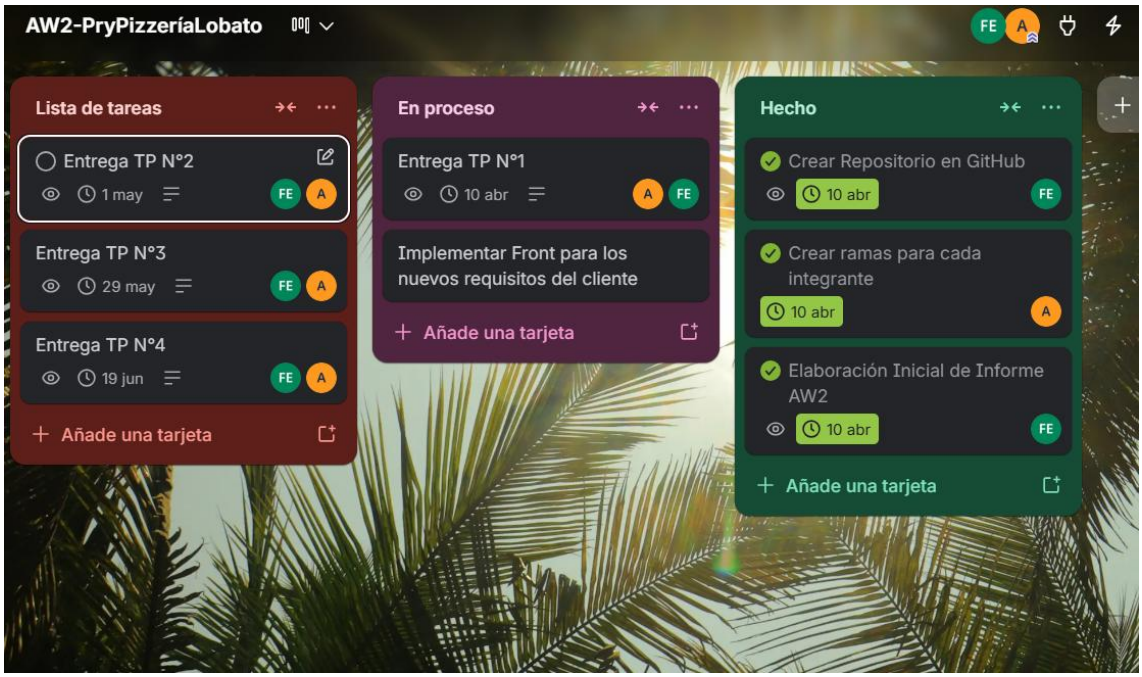
La existencia de este carrito implica también, la **posible eliminación** de la **sección** “**contacto**”, o su **integración** al mismo carrito.

Además de lo que vera el cliente, también se tiene pensado el desarrollo de una **gestión de usuarios**. La idea es que haya un administrador que pueda loguearse y poder: modificar los precios, imágenes, agregar o sacar productos.

4. Cronograma y organización de tareas (TP1)

Semana	Fecha	Desarrollo
1	23/03 al 27/03	Creación del repositorio GitHub.
2	30/03 al 03/04	Creación de las ramas con el nombre de cada integrante.
3	06/04 al 10/04	Elaboración de Informe y entrega del primer trabajo práctico.
4	13/04 al 17/04	Creación del servidor e instalación de nodeJS y expressJS.
5	20/04 al 24/04	Carga de datos en plataforma mockapi. Modularización del proyecto, desarrollo del informe.
6	27/04 al 01/05	Reunión presencial con el equipo, control de errores, rotación de roles y carga del html en servidor. Desarrollo del informe
7	04/05 al 08/05	Organización de proyectos utilizando patrón MVC.
8	11/05 al 15/05	Creación de APIs.
9	18/05 al 22/05	Creación interfaz front-end para administración.
10	25/05 al 29/05	Utilización de una base de datos. Reunión presencial del equipo para control de errores y rotación de roles. Desarrollo del informe.
11	01/06 al 05/06	Crear persistencia de datos en BBDD. Utilización del hashing.
12	08/06 al 12/06	Implementación de autenticación y control de acceso. Habilitar CORS en caso de ser necesario (Justificar).
13	15/06 al 19/06	Uso de variables de entorno, desarrollo del informe. Reunion presencial con el equipo y control de errores.

Se ha decidido como **herramienta** de seguimiento y organización de tareas utilizar **Trello**, debido a que ya existe cierta familiaridad en el uso de la misma.



Link: https://trello.com/b/6jXaEqUX/aw2-prypizzerialobato

5. Seguimiento y control de versiones (TP1)

Por último, se ha tomado la decisión de utilizar como gestor de versiones "**GITHUB**".

Link: https://github.com/Mariano-Lopez/TP-Lobato-AW2-Elgueta-Lopez

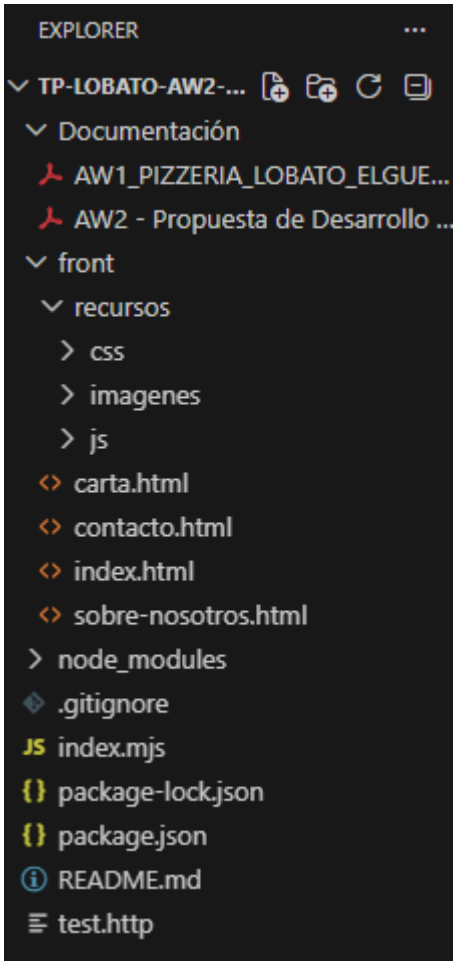
6. Back-end para servir el sitio web (TP2)

En esta etapa se desarrolló un servidor local, utilizando las tecnologías NodeJS y Express. Para lo cual se debió estructurar nuevamente el proyecto. Se creo una nueva carpeta “front” en la raíz, donde se guarda toda la interfaz del sitio, incluyendo HTML, CSS, JS e imágenes.

Nueva estructura del proyecto

A continuación, se detallará cada archivo y carpeta:

- **/index.mjs:** Archivo principal donde se ejecutará el servidor local y donde se importan los módulos *Express* y *path*.
- **/test.http:** Archivo donde se controla que los endpoints se ejecuten correctamente.
- **/README.md:** Archivo para guardar anotaciones.
- **/package.json:** Archivo creado por el modulo *Express*, donde se debe configurar el “type” en modules y en script configurar los identificadores al momento de ejecutar npm.
- **/package-lock.json:** Archivo creado por modulo *Express*.
- **/.gitignore:** Archivo git donde se especifica que carpeta debe ignorar al momento de subir los cambios en el repositorio, lo cual, en este caso, es la carpeta “node_modules”
- **front/recursos:** Contiene a su vez 3 carpetas con toda la configuración de CSS, JS y la última siendo la que almacena las imágenes del sitio completo.
- **front/index.html:** Archivo HTML de inicio del sitio.
- **front/carta.html:** Archivo HTML de la carta del sitio.
- **front/sobre-nosotros.html:** Archivo HTML con historia del negocio del sitio.
- **front/contacto.html:** Archivo HTML de contacto del sitio.



(nueva estructura del proyecto)

Tecnologías

Para el desarrollo de esta etapa se ha decidido utilizar las siguientes tecnologías:

- **NodeJS**: Entorno de ejecución de JavaScript de código abierto y multiplataforma que permite ejecutar código JS fuera del navegador web, principalmente en servidores.
- **Express**: Es el framework web estándar para Node.js, diseñado para crear aplicaciones web y APIs de forma rápida y flexible.
- **Path**: Es una utilidad integrada diseñada para gestionar y transformar rutas de archivos y directorios de forma segura entre diferentes sistemas operativos.
- **NPM**: Es el gestor de paquetes predeterminado para Node.js.

Datos de API externa (links mockApi)

- https://69e68ec268208c1debe7aecb.mockapi.io/pizzeria-lobato/productos
- https://69e68ec268208c1debe7aecb.mockapi.io/pizzeria-lobato/promociones

➢ Estructura de los datos

- **id**: Identificador único.
- **nombre**: Nombre del producto.
- **precio**: Monto del producto.
- **imagen**: Se divide nuevamente en 2 partes:
  - **src**: Ruta de archivo.
  - **alt**: En caso de no cargar imagen, texto descriptivo.
- **categoria**: Categoría a la que pertenece.

Desarrollo completo

El servidor como se mencionó anteriormente fue creado con modulo Express.

Se creó un puerto predeterminado para consultar, que en este caso es el 3000.

Se utiliza el método de express ".use" donde la búsqueda de la interfaz la realiza dentro de la carpeta *front*. Para acceder a las otras páginas del sitio se debe especificar la ruta exacta, por ejemplo: "http://localhost:3000/carta".

Se debe declarar que el servidor escuche al puerto creado anteriormente mediante parámetro, utilizando el método ".listen".

Cuando el cliente realiza la petición, cada modulo realiza el fetch correspondiente para devolver la información.